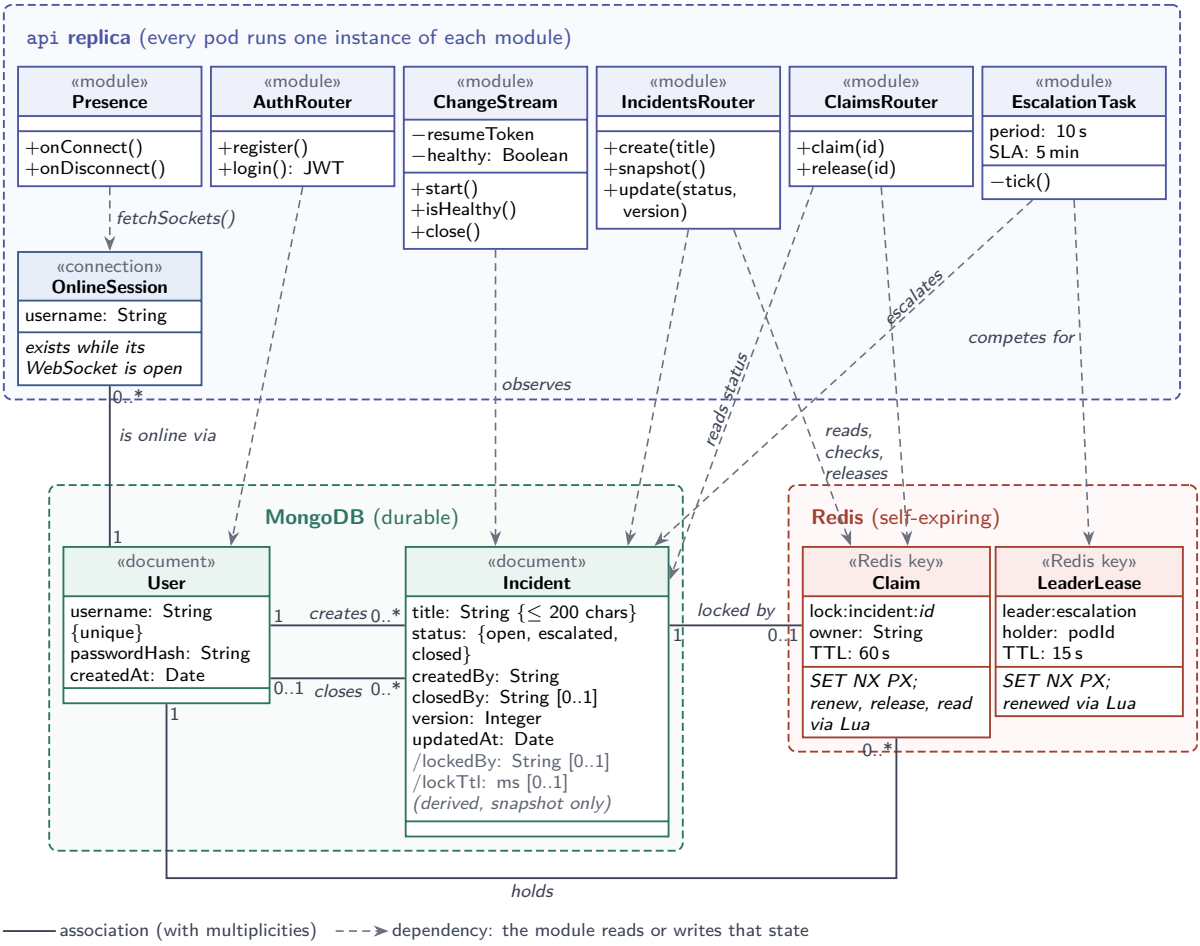

api replica (every pod runs one instance of each module)
«module» Presence
+onConnect()
+onDisconnect()
«module» AuthRouter
+register()
+login(): JWT
«module» ChangeStream
−resumeToken
−healthy: Boolean
+start()
+isHealthy()
+close()
«module» IncidentsRouter
+create(title)
+snapshot()
+update(status, version)
«module» ClaimsRouter
+claim(id)
+release(id)
«module» EscalationTask
period: 10 s
SLA: 5 min
−tick()
fetchSockets()
«connection» OnlineSession
username: String
exists while its WebSocket is open
0..*
is online via
1
observes
reads status
reads, checks, releases
escalates
competes for
MongoDB (durable)
«document» User
username: String {unique}
passwordHash: String
createdAt: Date
1
creates
0..*
0..1
closes
0..*
1
«document» Incident
title: String {≤ 200 chars}
status: {open, escalated, closed}
createdBy: String
closedBy: String [0..1]
version: Integer
updatedAt: Date
/lockedBy: String [0..1]
/lockTtl: ms [0..1]
(derived, snapshot only)
locked by
1
0..1
Redis (self-expiring)
«Redis key» Claim
lock:incident:id
owner: String
TTL: 60 s
SET NX PX; renew, release, read via Lua
«Redis key» LeaderLease
leader:escalation
holder: podId
TTL: 15 s
SET NX PX; renewed via Lua
0..*
holds
association (with multiplicities)
dependency: the module reads or writes that state